Python

Python cho người đã quen C++

Phase 0 · Bài 0.1

Từ C++ sang Python

Vòng lặp, hàm, mảng, độ phức tạp: các em đã có.

Mọi thư viện AI của khóa học được gọi từ Python.

Bài này chỉ ra những chỗ **khác C++ đủ để gây lỗi**.

CÂU HỎI

`-7 / 2` trong C++ là -3.

Trong Python, `-7 // 2` là bao nhiêu?

Sau buổi này, các em có thể

- Đoán đúng phép chia, lấy dư, so sánh số thực, kể cả với số âm.
- Giải thích vì sao sửa list chỗ này lại đổi list chỗ khác, và sao chép cho đúng.
- Chọn cấu trúc dữ liệu theo độ phức tạp, ước lượng thời gian chạy.
- Đọc, ghi CSV và đọc thông báo lỗi để tự sửa code.

Python để điều khiển, thư viện để tính

- Vòng lặp Python chậm hơn C++ cỡ 100 lần.
- `a.sum()` của NumPy, `model.fit()` của PyTorch chạy bằng C, C++, CUDA.
- Python là lớp keo: đọc dữ liệu, gọi thư viện, ghép các bước.

NOTEBOOK

Các ô dùng chung biến. Chạy lệch thứ tự hoặc sửa ô trên mà quên chạy lại là lỗi C++ không có.

Trước khi nộp: **Restart and run all**.

01

Số và phép toán

Số nguyên không tràn, chia làm tròn xuống

```
print(2 ** 63 - 1 + 1)
print(7 / 2, 6 / 2)
print(7 // 2, -7 // 2)
print(7 % 3, -7 % 3)
```

KẾT QUẢ

```
9223372036854775808
3.5 3.0
3 -4
1 2
```

- `/` luôn cho số thực.
- `//` làm tròn **xuống**, không về 0 như C++.
- `%` theo `//`: số chia dương thì kết quả không âm.
- `(i - 1) % n` luôn trong $[0, n)$.

Số thực vẫn là double

```
print(0.1 + 0.2, 0.1 + 0.2 == 0.3)
print(math.isclose(0.1 + 0.2, 0.3))
print(round(2.5), round(3.5))
```

KẾT QUẢ

```
0.30000000000000004 False
True
2 4
```

- Không so sánh số thực bằng ==.
- Dùng `math.isclose`, với mảng thì `np.isclose`.
- `round` làm tròn về số chẵn khi đúng ở giữa.

HỎI NHANH

Python in ra gì cho `print(-7 // 2, -7 % 2)`?

A `-3 -1`

B `-4 1`

C `-3 1`

D `-3.5 1`

02

Tên và đối tượng

Phép gán không sao chép

```
a = [1, 2, 3]
b = a
b.append(4)
print(a, b, a is b)
```

KẾT QUẢ

```
[1, 2, 3, 4] [1, 2, 3, 4] True
```

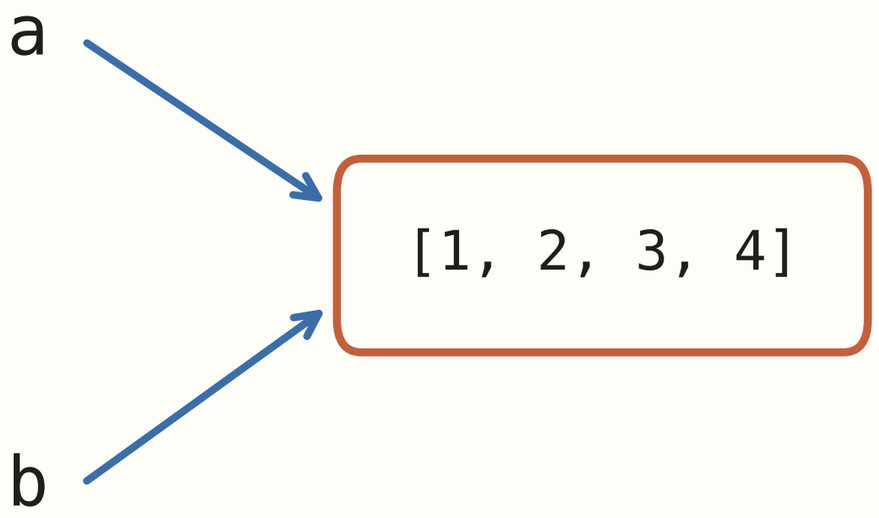
Biến là **cái tên** dán lên đối tượng, không phải cái hộp.

`is`: hai tên có chỉ **cùng một** đối tượng không.

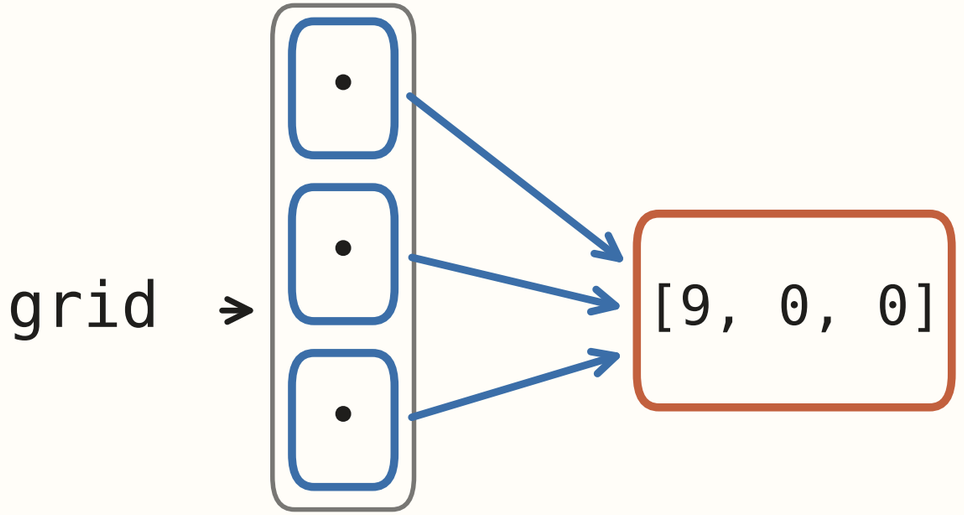
`==`: hai đối tượng có **bằng nhau** không.

Hai tên, một đối tượng

b = a → một list, hai tên



[[0] * 3] * 3 → một hàng, ba tham chiếu



Trái: b = a dán hai tên lên một list. Phải: một hàng được tham chiếu ba lần.

Bất biến và sửa được

Bất biến	Sửa được
`int`, `float`, `str`, `tuple`	`list`, `dict`, `set`, mảng NumPy

`y += 1` với số nguyên tạo đối tượng mới, nên `x` không đổi.

Hàm nhận list rồi sửa nó thì người gọi thấy thay đổi.

```
c = a.copy()          # hoặc a[:], list(a)
grid = [[0] * 3] * 3              # SAI
grid = [[0] * 3 for _ in range(3)]  # ĐÚNG
```

KẾT QUẢ

```
[[9, 0, 0], [9, 0, 0], [9, 0, 0]]
[[9, 0, 0], [0, 0, 0], [0, 0, 0]]
```

`copy()` chỉ sao một tầng; sâu hơn thì `copy.deepcopy`.

HỎI NHANH

`a = [[1, 2], [3, 4]]`, `b = a.copy()`, `b[0].append(9)`. `a` là gì?

A `[[1, 2], [3, 4]]`

B `[[1, 2, 9], [3, 4]]`

C `[[1, 2], [3, 4], 9]`

D Báo lỗi

03

Cấu trúc dữ liệu

STL trong Python

C++	Python	Ghi chú
`vector<T>`	`list`	`append` là $O(1)$
`pair`, `tuple`	`tuple`	Bất biến, làm khóa dict được
`unordered_map`	`dict`	Giữ thứ tự chèn, tra $O(1)$
`unordered_set`	`set`	`in` là $O(1)$
`map`, `set` có thứ tự	không có	Dùng `sorted()` khi cần

Slicing, comprehension, Counter

```
a = [5, 3, 8, 1, 9, 2]
print(a[1:4], a[-1], a[::-1])
print([x * x for x in a if x % 2 == 1])
print(Counter(words).most_common(2))
```

KẾT QUẢ

```
[3, 8, 1] 2 [2, 9, 1, 8, 3, 5]
[25, 9, 1, 81]
[('mèo', 3), ('chó', 2)]
```

- `sorted(items, key=lambda kv: -kv[1])` thay cho hàm so sánh.
- `d.get(k, 0)` tránh `KeyError`.

`in` trên list và set

1000 truy vấn vào 10^5 phần tử, trên máy soạn bài:

Cấu trúc	Thời gian
`list`	0.43 giây
`set`	0.00006 giây

List duyệt tuần tự $O(n)$, set băm $O(1)$.

Python khuếch đại cái giá của lựa chọn sai: chọn cấu trúc như khi thi lập trình.

04

Hàm và tốc độ

enumerate, zip, nhiều giá trị trả về

```
for i, (name, mark) in enumerate(zip(names, marks), start=1):
    print(f"{i}. {name}: {mark}")

def stats(xs, ddof=0):
    ...
    return mean, var

m, v = stats(marks)
```

THAM SỐ MẶC ĐỊNH LÀ LIST

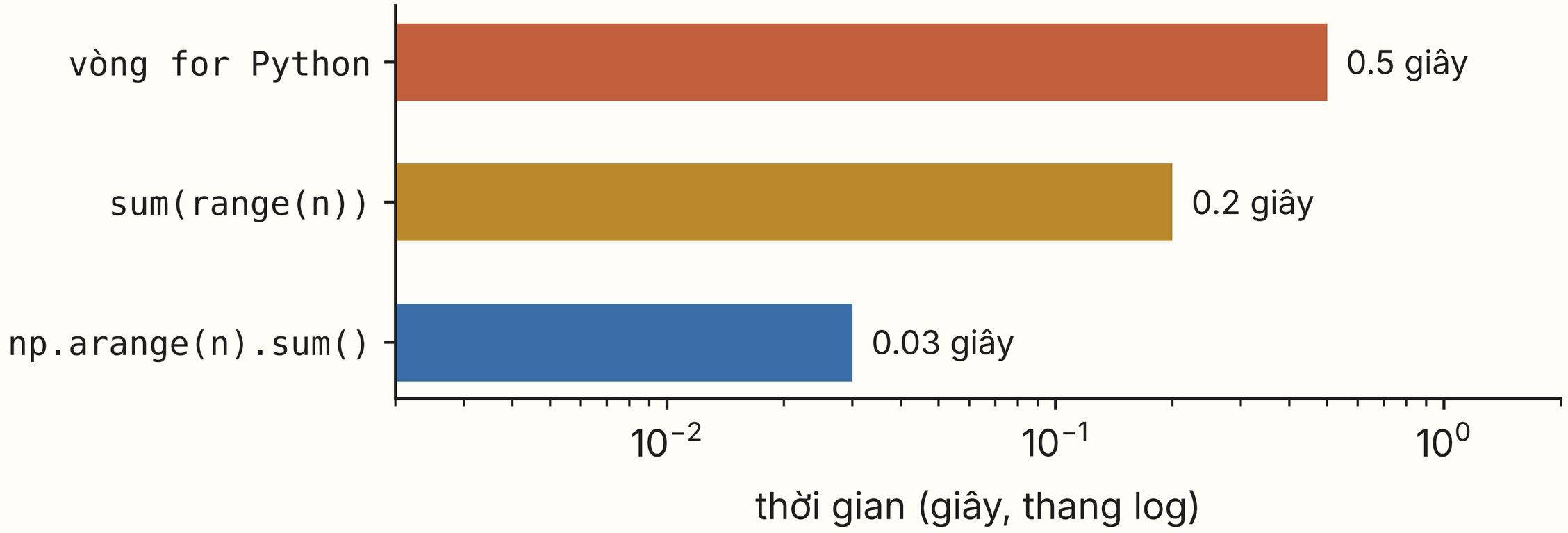
```
def add_score(x, history=[]):
    history.append(x)
    return history
print(add_score(1), add_score(2))
```

KẾT QUẢ

```
[1, 2] [1, 2]
```

List mặc định tạo **một lần**, dùng chung mọi lần gọi. Dùng `history=None`.

Tổng 10^7 số



Vòng for 0.5 giây, sum(range(n)) 0.2 giây, NumPy 0.03 giây.

Vòng lặp Python: cỡ 10^7 bước mỗi giây. Lặp trên dữ liệu lớn thì đẩy xuống thư viện.

HỎI NHANH

Hai vòng lặp lồng nhau, mỗi vòng 10^4 lần, thân vòng vài phép cộng. Python mất bao lâu?

A Dưới 0.1 giây

B Khoảng 1 giây

C Khoảng 10 giây trở lên

D Không chạy được

05

Chuỗi, file và lỗi

f-string và CSV

```
print(f"{name:>10} | accuracy = {acc:.3f} | {acc:.1%}")
with open(path, "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["id", "prediction"])
```

KẾT QUẢ

```
  logistic | accuracy = 0.854 | 85.4%
```

- Luôn ghi `encoding="utf-8"`: mặc định trên Windows không phải UTF-8.
- Mọi ô đọc từ CSV là **chuỗi**: `"10" < "9"`.
- Bài 0.3: pandas tự đoán kiểu từng cột.

Đọc traceback từ dưới lên

KẾT QUẢ

```
TypeError: can only concatenate str (not "int") to str
KeyError: 'Dũng'
IndexError: list index out of range
ValueError: invalid literal for int() with base 10: '7.5'
```

- Dòng cuối: chuyện gì xảy ra. Các dòng trên: ở đâu.
- Sai mà không báo lỗi: in kiểu, độ dài, vài phần tử đầu.
- `assert len(preds) == 3, len(preds)` dừng ngay khi giả định sai.

Đoán trước khi chạy

1. `a = [1, 2]; b = a; a = a + [3]; print(b)`
2. `print(10 // -3, 10 % -3)`
3. `s = "abc"; t = s; t += "d"; print(s)`
4. `d[[1, 2]] = "y"`

1. `[1, 2]`: `a + [3]` tạo list mới. (`a += [3]` thì sửa list cũ.)
2. `-4 -2`
3. `abc`: chuỗi bất biến.
4. `TypeError: unhashable type: 'list'`

Bẫy cài đặt

Bẫy	Hậu quả	Cách tránh
`sum = 0`, `max = 10`	`TypeError: 'int' object is not callable`	Đặt `total`, `best`; `del sum`
Xóa phần tử khi đang duyệt	Bỏ sót phần tử	Tạo list mới bằng comprehension
Chạy ô lệch thứ tự	Chạy lại từ đầu thì hỏng	Restart and run all
`is` thay cho `==`	So danh tính, không so giá trị	`is` chỉ dùng với `None`

Khi đi thi

- **Restart and run all** trước mỗi lần nộp.
- **Đặt seed** cho `random`, NumPy, PyTorch ở ô đầu tiên.
- **Không lặp Python trên từng mẫu** khi dữ liệu từ 10^5 dòng.
- `pathlib`, **đường dẫn theo đề** (`/kaggle/input/...`); in danh sách file trước khi đọc.
- `encoding="utf-8"`, đọc lại file nộp để kiểm tra.

Tóm tắt

- Python điều khiển, thư viện tính. Vòng lặp Python: cỡ 10^7 bước mỗi giây.
- Số nguyên không tràn; `//`, `%` làm tròn xuống; không so số thực bằng $=$.
- Biến là tên; phép gán không sao chép; `copy()`, `deepcopy`.
- Chọn list, dict, set theo độ phức tạp.
- `encoding="utf-8"`; CSV đọc ra chuỗi; traceback đọc từ dưới lên.

Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook của bài: đo thời gian `in` trên list và set với 10^3, 10^4, 10^5 phần tử, xem thời gian tăng thế nào.